



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



SERLAB
Software Engineering Research

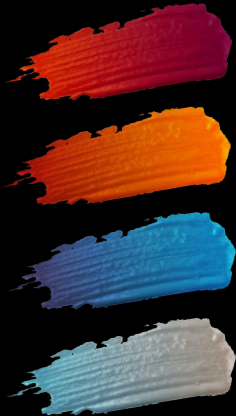
Ingegneria del Software

Introduzione

ITPS A.A. 2025/2026

Vita Santa Barletta
Danilo Caivano
Antonio Piccinno

Dipartimento di Informatica - Università degli Studi di Bari
Via Orabona, 4 - 70125 - Bari
Tel: +39.080.5443270 | Fax: +39.080.5442536
serlab.di.uniba.it



L'INGEGNERIA del SOFTWARE



Ingegneria del Software

- Agli albori il problema dell'informatica era scrivere le istruzioni per risolvere un problema
 - ❑ Il programmatore era l'utilizzatore stesso (ad es. fisico per calcoli scientifici)
- Con la diminuzione dei prezzi dei computer e loro diffusione fa aumentare gli utilizzatori: **programmare** diventa una professione
 - ❑ I programmatori scrivono programmi per altri
 - ❑ Separazione tra utente e programmatore
- L'utente **specifica** cosa vuole (nel suo linguaggio)
 - ❑ Il programmatore legge la specifica e la traduce in programma



Il Ruolo dell'Ingegnere del software

- L'ingegnere del Software non deve solo **programmare in piccolo**, egli è coinvolto nella **programmazione in grande**
- Per **PROGRAMMARE IN PICCOLO** deve essere:
 - ❑ Un buon programmatore
 - ❑ Esperto di strutture dati ed algoritmi
 - ❑ Conoscitore di uno o più linguaggi di programmazione
- Per **PROGRAMMARE IN GRANDE** deve essere in grado di:
 - ❑ **Sviluppare modelli** necessari nelle varie fasi dei processi di sviluppo per effettuare, grazie a questi, i compromessi necessari
 - ❑ **Esprimere i concetti** inerenti il software **con diversi livelli di astrazione**, dipendentemente dal destinatario del manufatto che produce
 - ❑ **Trovare le risorse utili** per accelerare ed economizzare il processo di sviluppo (componenti, pattern, template, esperienze ...)
 - ❑ **Lavorare in gruppo**
 - ❑ **Gestire progetti** e coordinare il lavoro degli altri



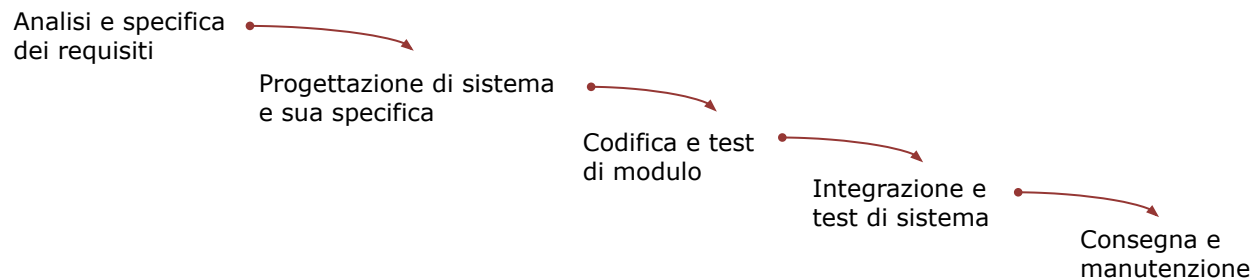
Risorse dell'Ingegnere del Software

- **Modelli per la caratterizzazione dei prodotti**, dei processi e delle risorse
- **Modelli per la stima dell'impegno di risorse** nei progetti di sviluppo software
- **Modelli per la valutazione ed il monitoraggio della qualità** dei prodotti e dei processi
- Nella I.S. questi modelli sono raramente basati su **tecniche matematiche**; molto spesso sono basati su **Esperienza** e sulla **Indagine Empirica**



Il ciclo di vita del software (1/3)

- Il software subisce uno sviluppo e un'evoluzione, dall'idea iniziale di un possibile prodotto o sistema software fino a quando viene implementato e consegnato al cliente
- Il software ha un **ciclo di vita** composto da **fasi**
 - ❑ Ad ogni fase è associato lo sviluppo di una parte del sistema o di qualche elemento a questo legato
 - ❑ Ogni fase ha un'inizio e una fine ben definita, con dei risultati parziali (artefatti) che vengono trasferiti a una ben identificata fase successiva
 - ❑ Modello tradizionale, chiamato **Modello a Cascata**



Il ciclo di vita del software (2/3)

Analisi e Specifica dei Requisiti

- Definisce in modo preciso e formale quali sono le capacità e le caratteristiche del Sistema Software da produrre.
- Produce anche
 - ☐ Manuali Utenti
 - ☐ Test di Accettazione

Progettazione di sistema e Specifiche

- Organizza le componenti che comporranno il sistema e le loro relazioni, attraverso **l'Architettura**
- Le **componenti** sono decomposte, a basso livello, in moduli.
- Produce anche
 - ☐ il **Manuale di Sistema**
 - ☐ il Test di **Integrazione**
 - ☐ il Test di **Sistema**



Il ciclo di vita del software (3/3)

Codifica e Test dei moduli

- Produce i moduli e ne fa il test per correttezza

Integrazione e Test di sistema

- Integra tutti i moduli prodotti secondo quanto previsto dal progetto ed esegue il **Test di Integrazione**
- Terminata l'integrazione esegue il **Test di Sistema**

Consegna e Manutenzione

- Quando il sistema supera anche il Test di Accettazione è consegnato al committente
- Durante l'esercizio del sistema esso è mantenuto per eliminare malfunzionamenti e per adeguarlo a nuove esigenze



Rapporto tra I.S. altri campi dell'Informatica

- **Linguaggi di programmazione**

- ☐ La I.S. influenza i linguaggi di programmazione che rendono sempre più servizi utili per strutturare meglio il software. Ad esempio i **pacchetti** in ADA e JAVA; le librerie di componenti; i linguaggi di interfaccia ...

- **Sistemi Operativi**

- ☐ Questi influenzano molto la I.S. perché essi sono grandi sistemi che hanno problemi che si incontrano molto spesso nelle grandi applicazioni

- **Le basi di dati**

- ☐ Hanno stimolato ed aiutato la I.S. nella realizzazione del principio di **separazione degli interessi**

- **Intelligenza Artificiale**

- ☐ Questa influenza ed è influenzata dalla I.S..
- ☐ Ha **portato nuove tecniche di definizione dei requisiti** in presenza di incertezza
- ☐ Ha invece **importato tecniche per la separazione degli interessi negli agenti**



Relazioni tra I.S. e altre discipline

- **Modelli teorici**

- ☐ Molti modelli teorici sono stati importati dalla I.S. Ad esempio protocolli di comunicazione e macchine a stati finiti. Altri modelli sono stati stimolati dalla I.S. ad esempio **specifiche algebriche** e **modelli di dati astratti**

- **Scienze Organizzative**

- ☐ La I.S. prende da queste discipline molti dei **modelli per la gestione di sistemi e processi complessi**.
- ☐ Stimola questa disciplina a studiare **nuovi modelli di stima e di gestione delle filiere di produzione**, visto che quelli che usa nelle produzioni materiali si adattano male alle produzioni incentrate sull'uomo

- **Ingegneria dei Sistemi**

- ☐ Ha relazione con la I.S. sia perché studia sistemi complessi sia perché il **software è sempre una componente di sistemi più grandi**



IL VALORE CENTRALE DELL'INGEGNERIA DEL SOFTWARE: LA QUALITÀ



Dal processo al prodotto

- Nell'ambito della **qualità del software** si sta assistendo nell'ultimo decennio ad un *processo di convergenza virtuoso e progressivo* tra
 - ❑ **QUALITÀ DI PROCESSO**
 - ❑ **QUALITÀ DI PRODOTTO**
- Il processo di produzione del software consiste essenzialmente nel progetto e nell'implementazione
 - ❑ Questo processo deve soddisfare opportuni criteri che assicurino la produzione di software di elevata qualità
 - ❑ È auspicabile che un prodotto soddisfi determinate necessità e rispetti standard di accettazione che prescrivono le qualità che deve possedere



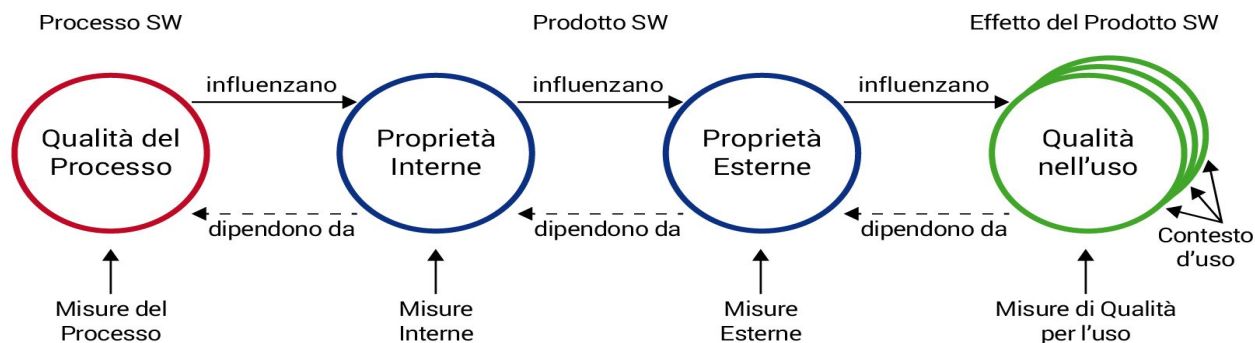
Dal processo al prodotto

- Dagli anni 80 circa, se qualcosa nel prodotto finale andava più o meno bene derivava da quello che era stato non fatto o fatto più o meno bene durante il processo di produzione
 - ❑ La qualità del prodotto software è stata perseguita con la **proposizione/definizione di processi altamente strutturati, documentati e controllati quantitativamente** durante l'esecuzione
 - ❑ Ciò anche perché un controllo di qualità operato direttamente sul prodotto software che è immateriale e di natura uomo centrica sarebbe impossibile
- Negli ultimi anni sono state sviluppate linee guida ed euristiche a supporto dello sviluppo del software che consentono oggi di **approcciare la qualità del software da prospettive diverse** affiancando alla classica attenzione rivolta al processo quella rivolta direttamente al prodotto software



Le tre prospettive della qualità

- Una **caratteristica di qualità** è una proprietà desiderabile che il prodotto/processo deve possedere per soddisfare requisiti impliciti o espliciti
- Nella famiglia di standard ISO 25000, la tendenza è quella di far convergere la valutazione verso un modello integrato basato su 3 punti di vista:
 - ❑ **QUALITÀ IN USO** (percepita): esprime l'efficacia ed efficienza con cui il software serve le esigenze dell'utente, ed è correlata alla percezione diretta dell'utente
 - ❑ **QUALITÀ INTERNA**: esprime la misura in cui il codice software possiede una serie di attributi statici, indipendentemente dall'ambiente di utilizzo e dall'utente
 - ❑ **QUALITÀ ESTERNA**: esprime il comportamento dinamico del software, nell'ambito d'uso
- L'assunto è che i tre punti di vista si influenzano a vicenda e che non può esservi qualità percepita positivamente dall'utente senza che vi sia una buona qualità intrinseca al codice e buone prestazioni



Caratteristiche di qualità del software

- **Correttezza**

- ❑ Un sistema software deve soddisfare tutti i suoi requisiti funzionali e di prestazione
- ❑ Un programma è funzionalmente corretto se si comporta secondo quanto stabilito dalla sue specifiche funzionali
- ❑ La definizione di correttezza assume che le specifiche del sistema siano disponibili e che sia possibile determinare in maniera non ambigua se un programma soddisfi le specifiche

- **Affidabilità**

- ❑ Probabilità che un sistema software si comporti secondo le attese in un intervallo di tempo

- **Robustezza**

- ❑ Un sistema software si deve comportare in modo accettabile anche in circostanze non previste nelle sue specifiche



Principali qualità del software

- **Usabilità**

- Un sistema software deve essere reputato di facile uso dai suoi utilizzatori

- **Manutenibilità**

- Facilità con cui:
 - si identifica la causa di un malfunzionamento e si elimina [**RIPARABILITÀ**]
 - si aggiungono, sottraggono e si modificano capacità e funzioni del sistema [**EVOLVIBILITÀ**]

- **Riusabilità**

- La possibilità di riusare delle componenti software o artefatti di processo software
 - La riusabilità può essere applicata, pertanto, a vari livelli di granularità
 - La programmazione object-oriented ha lo scopo di raggiungere sia una migliore riusabilità che una migliore evolvibilità

Principali qualità del software

- **Prestazioni (1 di 2)**

- ❑ **Tempo di risposta:** *tempo necessario* per elaborare un processo, ad esempio da quando si preme il tasto di avvio a quando mostra il risultato
- ❑ **Reattività:** tempo che impiega il sistema a *notificare* che ha acquisito la richiesta dell'utente è indipendente dal tempo di risposta
- ❑ **Latenza:** il *tempo minimo* necessario per ricevere un qualunque tipo di risposta, compresa la notifica che l'applicazione non è riuscita ad eseguire nulla
- ❑ **Throughput:** *quanto lavoro* il sistema riesce ad eseguire nell'unità di tempo: transazioni per secondo; byte trasferiti per secondo

Principali qualità del software

- **Prestazioni (2 di 2)**

- ❑ **Carico:** *quanto lavoro contemporaneo può fare il sistema: il numero di utenti che possono lavorare contemporaneamente*
- ❑ **Sensibilità al carico:** *quanto varia una prestazione con il cambiamento di carico*
- ❑ **Efficienza:** *la prestazione rapportata alle risorse utilizzate: throughput/numero di CPU*
- ❑ **Capacità**
 - *il massimo throughput o carico che un sistema può reggere*
 - *può essere un massimo assoluto od una soglia oltre la quale le prestazioni del sistema possono decadere sensibilmente*
- ❑ **Scalabilità:** *misura della capacità di modificare le prestazioni del sistema con l'aumento delle risorse disponibili*



Principali qualità del software

- **Verificabilità**

- ☐ È importante poter facilmente procedere alla verifica della correttezza e delle prestazioni di un sistema software
- ☐ La verifica potrà essere svolta attraverso metodi formali o informali, oppure attraverso test

- **Portabilità**

- ☐ Il software è portabile se può essere eseguito in ambienti diversi (piattaforma hardware o ambiente software)

- **Comprensibilità**

- ☐ La comprensibilità di un sistema software dipende in parte da come il software è progettato, ma in larga misura dipende dal problema affrontato dal software



Principali qualità del software

- **Interoperabilità**

- ☐ La capacità di un sistema di coesistere e cooperare con altri sistemi

- **Produttività**

- ☐ È una qualità del processo di produzione del software, ne indica l'efficienza e le prestazioni

- **Tempestività**

- ☐ È una qualità del processo che indica la capacità di rendere disponibile un prodotto al momento giusto

- **Visibilità**

- ☐ Un processo di sviluppo del software è visibile se tutti i passi successivi (step) e lo stato attuale sono documentati in modo chiaro

Impatto economico della Qualità del Software

- I fattori della qualità hanno influenza sui seguenti aspetti economici:
 - ❑ Il **costo** di produzione, manutenzione, evoluzione e supporto
 - ❑ La **pianificazione efficace** ed **efficiente** per produzione, manutenzione, evoluzione e supporto
 - ❑ I **ricavi diretti** che genera se venduto
 - ❑ I **ricavi indiretti** che genera dai servizi e dai prodotti collaterali
 - ❑ La **curva di apprendimento** per gli utilizzatori ed i consumatori
 - ❑ Il **risparmio dei costi operativi** che genera nelle organizzazioni utilizzatrici
 - ❑ Le **opportunità di nuovi tipi di affari** che l'applicazione genera alle organizzazioni utilizzatrici



I TIPI DI PRODOTTI SOFTWARE



Il Software

PROGRAMMA

- È un **insieme di istruzioni** autoconsistenti rispetto ad uno o più obiettivi
- Spesso è **usato dallo stesso** autore
 - ❑ È difficile farlo utilizzare da altri perché per **scarsa documentazione** è difficilmente comprensibile
- I suoi difetti sono **rilevati**, in genere, SUL CAMPO perché è **scarsa la validazione** durante la sua produzione
- Ha vita relativamente breve perché la manutenzione fa decadere la sua qualità e diventa **sempre più difficile e costoso farlo evolvere**

Il Software

APPLICAZIONE

- È un **insieme di programmi** interagenti tra loro
- Spesso è venduto come un **pacchetto** usabile da persone che non hanno dimestichezza con l'informatica perchè sono forniti, almeno, di un'interfaccia e di documentazione d'uso
- I loro **difetti** sono scoperti essenzialmente dagli utilizzatori, ma durante la produzione **una parte** sono scoperti attraverso la **validazione**
- Spesso hanno **bassi livelli di qualità**
 - ☐ Sono poco attrezzati per il trasferimento a nuovi sviluppatori
 - ☐ La qualità decade rapidamente
 - ☐ La loro manutenzione diventa sempre più costosa e rischiosa

Il Software

SISTEMA SOFTWARE

- È un **insieme di programmi interagenti** che copre un Dominio Applicativo con
 - ❑ **Alto livello di qualità**
 - ❑ Completo di tutta la **documentazione**
 - i **requisiti**
 - la **progettazione** che spiega la sua struttura e le decisioni che hanno giustificato la sua strutturazione
 - i **manuali** d'uso
- Per far rientrare gli **alti costi di produzione**, essi sono destinati ad un **esteso bacino di utenza** anche con piattaforme diverse
 - ❑ L'**usabilità** e la **portabilità** sono caratteristiche chiave
- Per poter essere redditizio, un sistema software deve avere una **lunga vita** e deve **invecchiare lentamente**



Applicazioni per l'Impresa

- Applicazioni per l'impresa sono **Sistemi Software** caratterizzati da:
 - ❑ **Dati persistenti**, necessari per passare le informazioni tra differenti applicazioni e tra differenti esecuzioni della stessa applicazione
 - ❑ **Rilevanti volumi di dati**, un sistema di medie dimensioni potrebbe avere diversi GB di dati organizzati in decine di tipi di records ed in milioni di records
 - ❑ **Accesso concorrente ai dati**, da diversi utilizzatori dell'applicazione
 - ❑ **Rilevante numero di schermate componenti l'interfaccia**, è frequente avere interfacce con centinaia di schermate
 - ❑ **Integrazione tra applicazioni che supportano domini differenti della stessa impresa**, molto spesso costruite con approcci eterogenei

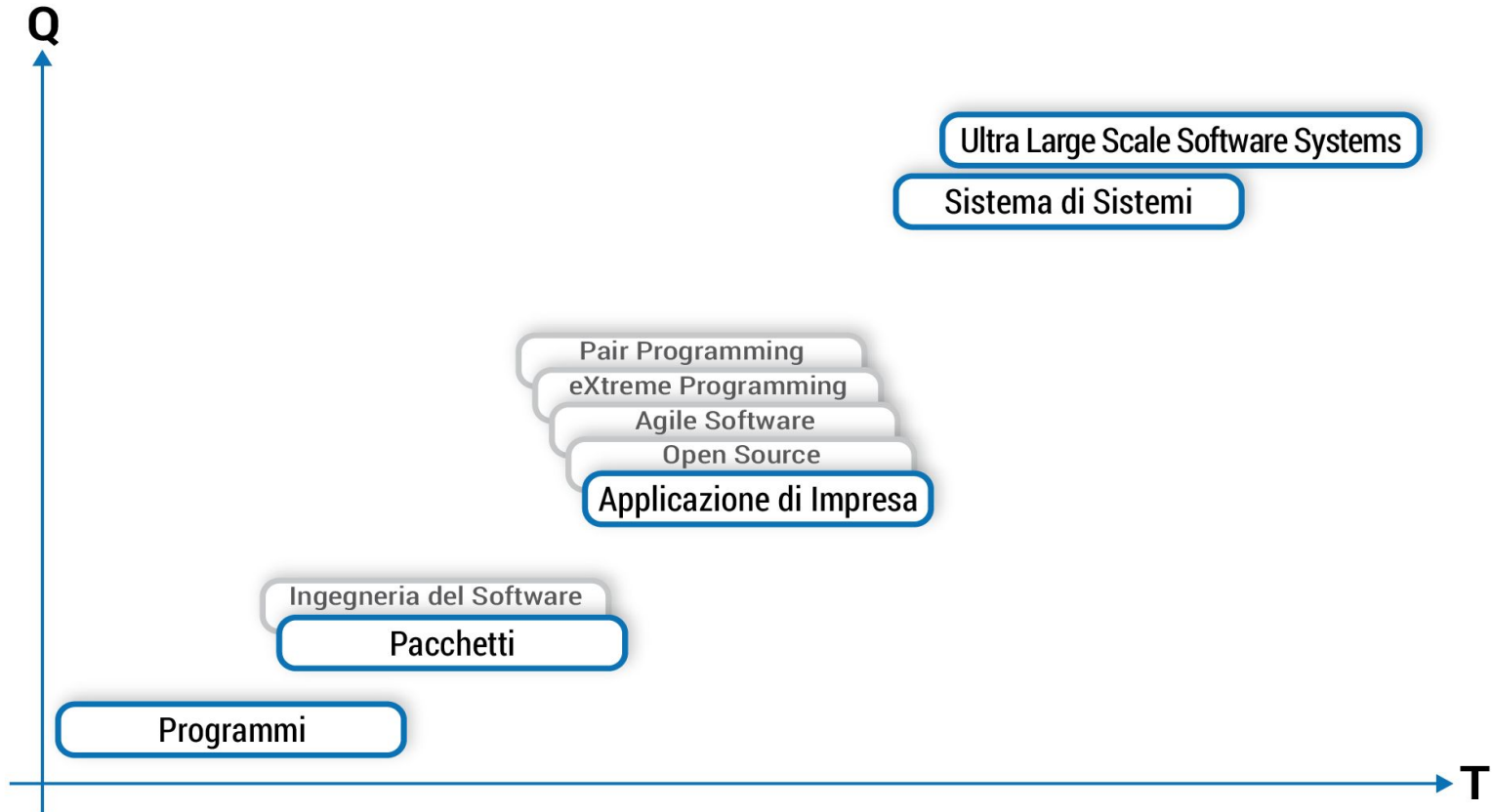
Sistemi in Tempo Reale

- La caratteristica principale è la **risposta** a particolari condizioni esterne **entro un predeterminato intervallo di tempo** che in genere è piccolo
- Sono **orientati al controllo**
 - ❑ In genere gestiscono pochi dati e molte funzioni
- Possono essere basati su **eventi** oppure basati sul **tempo**
- Spesso sono utilizzati per operazioni critiche quindi devono avere caratteristiche di **affidabilità** e di **salvaguardia/protezione** (safety) in condizioni di rischio

Sistemi Embedded

- Sistemi componenti di altri sistemi che spesso **non hanno interfacce verso gli utenti esterni**
- **Hanno interfacce verso dispositivi** dello stesso sistema di cui essi sono componenti.
- In genere hanno **pochi dati** e **poche capacità funzionali**

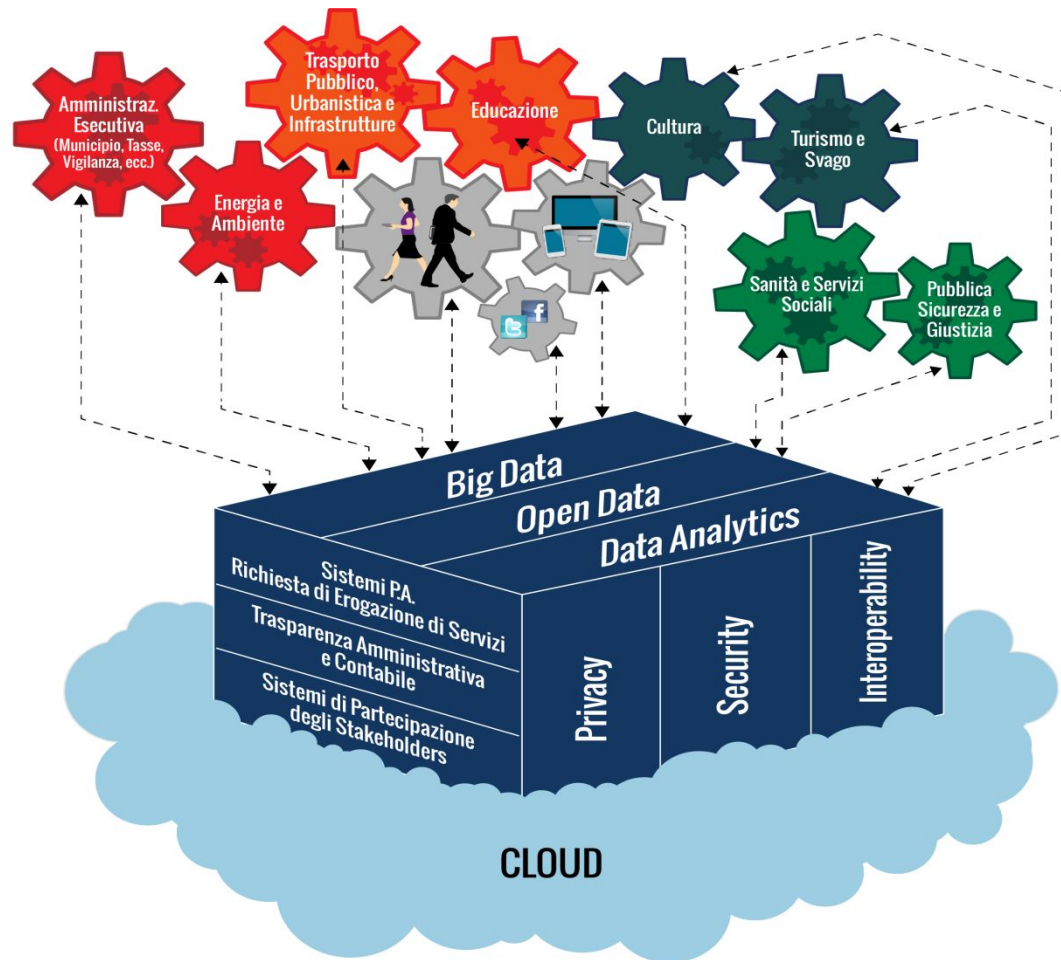
Il software nel tempo



Sistemi Ultra Grandi (ULSS): Smart System



Impatto dei Sistemi Ultra Grandi



PROBLEMI DELLA INGEGNERIA DEL SOFTWARE



Dissonanze Concettuali

- Molti concetti del Dominio Applicativo sono interpretati in modo diverso da utenti e da applicazioni diversi
 - ❑ Per esempio, un **cliente** può essere considerato:
 - un soggetto con cui l'impresa ha una relazione commerciale corrente, oppure
 - un soggetto con cui si è intrattenuta una relazione commerciale, anche se tale relazione non è stata mantenuta
- I **concetti** utilizzati in una applicazione devono essere **definiti con rigore**:
 - ❑ Facendo riferimento a conoscenza esplicita, per esempio: libri di testo, articoli, decreti legislativi, norme; oppure dichiarando la definizione in un **glossario**
 - ❑ quando uno stesso concetto in un dominio applicativo **può assumere diversi significati**, per consuetudine, allora l'**applicazione** deve essere **flessibile** in modo da essere compatibile con il significato che desidera ogni suo utente



Struttura Complessa

- Un **Dominio Applicativo** include, in genere, un insieme di **Processi di Business** che hanno **molte** relazioni tra loro
- Le **Applicazioni d'Impresa** devono tener conto delle **molte interrelazioni** tra i processi di business e perciò risultano essere molto complesse



Carenza di conformità

- La **conformità** esprime la corrispondenza delle attività eseguite, dei metodi, delle tecniche e dei strumenti utilizzati in un processo software con gli omologhi presenti nella sua definizione formale
- La **carenza di conformità** dei processi software causa **carenza di qualità**
 - ☐ dei **processi**
 - ☐ dei **prodotti**
- È necessaria la raccolta ed il trasferimento agli sviluppatori di **evidenze sperimentali** circa
 - ☐ la relazione tra questi principi e la economicità di costruzione, distribuzione e manutenzione del software



I PROCESSI SOFTWARE



Tendenze del software e Conseguenze

- I contesti applicativi oggi si caratterizzano per rapidi e crescenti evoluzioni a causa di:
 - ❑ Molta **diversità** nei requisiti per la numerosità degli utilizzatori
 - ❑ Crescente ritmo di **variazione** dei requisiti per la volatilità dei mercati
 - ❑ Maggiore **rapidità** di comprensione dei principi sottostanti il Dominio Applicativo per la numerosità degli attori coinvolti e la frequenza d'uso dei Sistemi di automazione
 - ❑ Crescente ritmo dei **cambiamenti** tecnologici per la pressione competitiva dei produttori
- La conseguenza che emerge è l'inadeguatezza dei processi ad approccio sequenziale
 - ❑ Ad esempio il classico Waterfall



Tecniche di produzione del software

Le tecniche di produzione per il rapido adeguamento dei processi si possono dividere in **due classi**:

- Tecniche basate sull'**ARCHITETTURA**
 - ❑ Si focalizzano sulle sorgenti di cambiamento più probabili o i requisiti più instabili e usano *l'information hiding* e la *modularità* per nascondere le sorgenti di cambiamento nei moduli dell'architettura
- Tecniche basate sul **REFACTORING**
 - ❑ Si focalizzano sul mantenere il prodotto il più semplice possibile e *adeguarlo incrementalmente* con successivi insiemi di cambiamenti dei requisiti



Scelta della classe più appropriata

- Se l'evoluzione dei requisiti si può conoscere per tempo e i **contenuti restano stabili** per un tempo sufficiente
 - ❑ Le tecniche basate sull'*architettura* sono adeguate
 - ❑ Quelle basate sul refactoring sarebbero costrette ad un eccessivo numero di ricicli, molti dei quali inutili
- Se i **cambiamenti sono frequenti** e difficilmente prevedibili o del tutto imprevedibili
 - ❑ Le soluzioni pre-architettate saranno continuamente infrante con relativi enormi costi di riprogettazione, quindi sono più adeguate le tecniche basate sul *refactoring*



I modelli di processo

- Le tecniche basate sull'architettura sono realizzate meglio con i **Processi guidati da piani:**
 - ❑ Rational Unified Process
 - ❑ Spirale
 - ❑ Waterfall e relative modifiche
 - ❑ ...
- Le tecniche basate sul refactoring sono realizzate meglio con i **Processi Agili:**
 - ❑ SCRUM
 - ❑ Agile
 - ❑ Adaptive Software Development
 - ❑ Extreme programming
 - ❑ Feature Driven Development
 - ❑ ...
- Si possono **combinare** modelli di processo guidati dai piani con quelli agili



Riferimenti

- Carlo Ghezzi, Medhi Jazayeri, Dino Mandrioli “Ingegneria del Software - Fondamenti e Principi, 2a edizione” Pearson Prentice Hall, 2004.
 - ❑ Capitolo 1: Ingegneria del software: visione d’insieme
 - ❑ Capitolo 2: Il software: natura e qualità

